

Roles y rutas protegidas

5. Roles y rutas protegidas

Al terminar el apartado anterior, tu API ya distinguía quién eres (JWT) de qué puedes hacer (roles) — pero solo lo habías comprobado sobre una ruta suelta, y ese 403 seguía saliendo con el formato por defecto de Spring. Hoy conviertes esa comprobación puntual en una política completa, ruta a ruta y cerrada por defecto.

La política de autorización, leída como tabla

Ruta	Verbo	Quién puede
/auth/register, /auth/login	POST	Cualquiera
/auth/me	GET	Cualquiera autenticado — cualquier rol vale
/libros, /editoriales	GET	Cualquiera
/libros, /editoriales	POST/PUT/DELETE	Solo ADMIN
/swagger-ui/**, /v3/api-docs/**, /error	—	Cualquiera
Cualquier otra ruta no listada	—	Nadie

authorizeHttpRequests, ruta a ruta

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(POST, "/api/v1/auth/register", "/api/v1/auth/login").permitAll()
    .requestMatchers(GET, "/api/v1/auth/me").authenticated()
    .requestMatchers(GET, "/api/v1/libros", "/api/v1/libros/**").permitAll()
    .requestMatchers(GET, "/api/v1/editoriales", "/api/v1/editoriales/**").permitAll()
    .requestMatchers(POST, "/api/v1/libros").hasRole("ADMIN")
    .requestMatchers(PUT, "/api/v1/libros/*").hasRole("ADMIN")
    .requestMatchers(DELETE, "/api/v1/libros/*").hasRole("ADMIN")
    .requestMatchers(POST, "/api/v1/editoriales").hasRole("ADMIN")
    .requestMatchers(PUT, "/api/v1/editoriales/*").hasRole("ADMIN")
    .requestMatchers(DELETE, "/api/v1/editoriales/*").hasRole("ADMIN")
    .requestMatchers("/error").permitAll()
    .requestMatchers("/v3/api-docs/**", "/swagger-ui/**", "/swagger-ui.html").permitAll()
    .anyRequest().denyAll()
)
```

Por qué /error va siempre en permitAll()

/error es la ruta interna que usa Spring Boot para construir la respuesta cuando una petición termina en un error que no ha capturado ningún `@ExceptionHandler`: Spring reenvía la petición, por dentro, a /error. Si esa ruta no es pública, ese reenvío también pasa por tu política de seguridad — y puedes acabar viendo un 401/403 de tu propia `denyAll()` en vez del error real.

La línea más importante de todo el bloque

```
.anyRequest().denyAll()
```

Cualquier ruta que no aparezca explícitamente en las reglas anteriores queda cerrada por defecto — es el principio de mínima exposición del primer apartado del tema, llevado hasta el final: nada se abre "por accidente" simplemente por existir.

La tabla no es una ley universal

Que POST/PUT/DELETE sobre /libros sean solo para ADMIN no es una regla fija de Spring Security: es la decisión correcta para este dominio, donde el catálogo lo gestiona el equipo, no los usuarios.

En una red social, un DELETE sobre una publicación normalmente sí lo puede hacer un USER, siempre que sea el autor — algo que ni siquiera se resuelve con un rol, sino comprobando de quién es el recurso. La pregunta que importa no es "¿qué verbo es?", sino "¿qué tiene sentido para lo que hace este endpoint, en esta aplicación concreta?".

Cada ruta nueva necesita su propia regla

⚠ O `denyAll()` la bloquea

Con `.anyRequest().authenticated()` como tenías hasta ahora, cualquier ruta sin regla propia funcionaba con solo estar autenticado. Al cambiar el catch-all a `denyAll()`, eso deja de ser cierto: lo que no tenga su línea explícita queda bloqueado, aunque el endpoint funcione perfectamente y mandes un token válido — "me da 403/401 sin motivo aparente" casi siempre significa "me he dejado esta ruta sin regla".

Ojo también con subrutas de una acción concreta (POST `/libros/5/aplicar-descuento`): no coinciden con la regla de la ruta base y necesitan la suya propia.

401 vs. 403 vs. 404, con criterio

Código	Significa	Cuándo aparece
401 Unauthorized	No sabes quién eres	Falta el token, o es inválido/caducado.
403 Forbidden	Sabemos quién eres, pero no puedes hacer esto	Token válido, pero rol insuficiente.
404 Not Found	La ruta pedida no corresponde a ningún endpoint	Pasa tu política de seguridad, pero no hay controller que la atienda.

El mismo endpoint, dos respuestas distintas

Con la tabla de antes: un POST /api/v1/libros sin token da 401; el mismo POST con el token de un usuario USER (no ADMIN) da 403. El 404 es distinto a los otros dos: no depende de quién eres, sino de si ese endpoint existe de verdad.

denyAll() sin token también da 401, no 403

denyAll() rechaza siempre cualquier petición que llegue hasta esa regla. Después, Spring Security distingue quién ha hecho la petición: si no existe una autenticación válida, ExceptionTranslationFilter llama al AuthenticationEntryPoint y devuelve 401; si la petición lleva un token válido, delega en el AccessDeniedHandler y devuelve 403.

Por tanto, la misma regla denyAll() puede terminar en uno u otro código: la diferencia no está en la regla, sino en si el solicitante está autenticado.

Cuando el 404 no es un 404

Pide una ruta que no existe pero coincide con una regla que la permite —GET /api/v1/libros/999/no-existe (dos segmentos tras la base, cae dentro de /api/v1/libros/**, que es pública)—. Deberías ver un 404...

...pero lo que sale es un 500 Error interno, generado por tu propio `GlobalExceptionHandler`.

No cualquier ruta "inexistente" sirve para este ejemplo

⚠ Un solo segmento coincide con `{id}`

Si en vez de dos segmentos pruebas con uno solo —GET `/api/v1/libros/esto-no-existe`—, esa ruta sí coincide con GET `/api/v1/libros/{id}`: Spring intenta convertir "esto-no-existe" al Long id, y lanza una excepción de conversión de tipo distinta (`MethodArgumentTypeMismatchException`), no `NoResourceFoundException`. Para provocar de verdad una ruta que no existe necesitas algo que no coincida con ningún patrón, ni por accidente.

NoResourceFoundException: un handler más específico

Cuando ninguna ruta coincide, Spring lanza `NoResourceFoundException` — y como también es una `Exception`, el Handler 5 genérico la atrapa antes de que Spring pueda darle su 404 correcto:

```
@ExceptionHandler (NoResourceFoundException.class)
public ResponseEntity<ErrorResponse> handleNoResourceFoundException(
    NoResourceFoundException ex, HttpServletRequest request) {

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), 404, "Not Found",
        "El recurso solicitado no existe", request.getRequestURI());
    return ResponseEntity.status(HttpStatus.NOT_FOUND).body(response);
}
```

Spring elige automáticamente el handler más específico para cada excepción — sin que tengas que tocar el orden de nada.

Un id que no es un id

GET /api/v1/libros/no-es-un-numero (un solo segmento, coincide con GET /api/v1/libros/{id}). Deberías ver un 400 —el id no tiene el formato correcto—, pero lo que sale es otra vez un 500 Error interno.

Spring intenta convertir el segmento de la URL al tipo esperado (Long id), no puede, y lanza `MethodArgumentTypeMismatchException`. Como tampoco tiene handler propio, cae otra vez en el Handler 5 genérico.

El mismo patrón exacto

```
@ExceptionHandler(MethodArgumentTypeMismatchException.class)
public ResponseEntity<ErrorResponse> handleMethodArgumentTypeMismatchException(
    MethodArgumentTypeMismatchException ex, HttpServletRequest request) {

    ErrorResponse response = new ErrorResponse(
        LocalDateTime.now().toString(), 400, "Parámetro inválido",
        "El valor '%s' no es válido para el parámetro '%s'".formatted(
            ex.getValue(), ex.getName()),
        request.getRequestURI());
    return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(response);
}
```

`ex.getValue()` da el valor que ha llegado y `ex.getName()` el nombre del parámetro — el mensaje dice exactamente qué ha fallado, sin adivinarlo.

Cerrando la última grieta: AccessDeniedHandler

El 403 de la tabla tampoco pasa por tu `GlobalExceptionHandler` —lo genera un filtro de seguridad, igual que el 401—, así que sale con el formato por defecto de Spring.

Spring Security tiene su propia pieza para este trabajo: un `AccessDeniedHandler`, la contraparte exacta de `AuthenticationEntryPoint` —mismo patrón, mismo momento del ciclo, pero para "sí sé quién eres, no puedes hacer esto" en vez de "no sé quién eres".

ErrorResponseAccessDeniedHandler

```
@Component
@RequiredArgsConstructor
public class ErrorResponseAccessDeniedHandler implements AccessDeniedHandler {

    private final ObjectMapper jsonMapper;

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
                      AccessDeniedException accessDeniedException) throws IOException {

        ErrorResponse error = new ErrorResponse(
            LocalDateTime.now().toString(), HttpStatus.FORBIDDEN.value(),
            "No autorizado", "No tienes permisos suficientes...", request.getRequestURI());

        response.setStatus(HttpStatus.FORBIDDEN.value());
        response.setContentType(MediaType.APPLICATION_JSON_VALUE);
        response.getWriter().write(jsonMapper.writeValueAsString(error));
    }
}
```

Los dos conviven en el mismo bloque

```
.exceptionHandling(exceptions -> exceptions
    .authenticationEntryPoint(customAuthenticationEntryPoint)
    .accessDeniedHandler(customAccessDeniedHandler)
)
```

Con esto, los tres códigos de la tabla —401, 403 y 404— tienen ya el mismo formato `ErrorResponse` que el resto de tu API, sin excepciones sueltas por ningún sitio.

Documentación como buena práctica

Depurar y documentar van de la mano. La propia tabla de política de rutas que has visto al principio ya es esa documentación — un documento vivo que cualquiera puede consultar para saber exactamente qué puede hacer cada rol, sin leer el código Java línea a línea. Mantener esa tabla al día, en un fichero aparte (por ejemplo docs/seguridad.md), es exactamente el tipo de documentación que se espera de ti.

Ideas clave

- La política de autorización se lee mejor como tabla (ruta × verbo × quién puede) que como código suelto.
- No existe una tabla verbo → rol universal: cada ruta se decide según lo que hace y para quién existe, no por convención.
- `anyRequest().denyAll()` cierra por defecto cualquier ruta sin regla explícita — cada ruta nueva necesita su propia regla o queda bloqueada, incluidas rutas que ya funcionaban con `authenticated()` y subrutas de acciones concretas.
- 401 = no sabemos quién eres; 403 = sabemos quién eres pero no tienes permiso; 404 = la ruta no existe. `denyAll()` sin token también da 401, no 403.

Ideas clave (cont.)

- `NoResourceFoundException` (ruta inexistente) y `MethodArgumentTypeMismatchException` (id con el tipo equivocado) caían en el Handler 5 genérico, dando 500 en vez de 404/400 — un handler específico lo arregla, sin tocar nada más.
- `AccessDeniedHandler` es la contraparte de `AuthenticationEntryPoint` para el 403: mismo patrón, registrado en el mismo `.exceptionHandling(...)`.
- La tabla de política de rutas es, en sí misma, esa documentación.